

ENGENHARIA DE SOFTWARE II

Prof. Júlio Machado

MICROSSERVIÇOS

Padrões

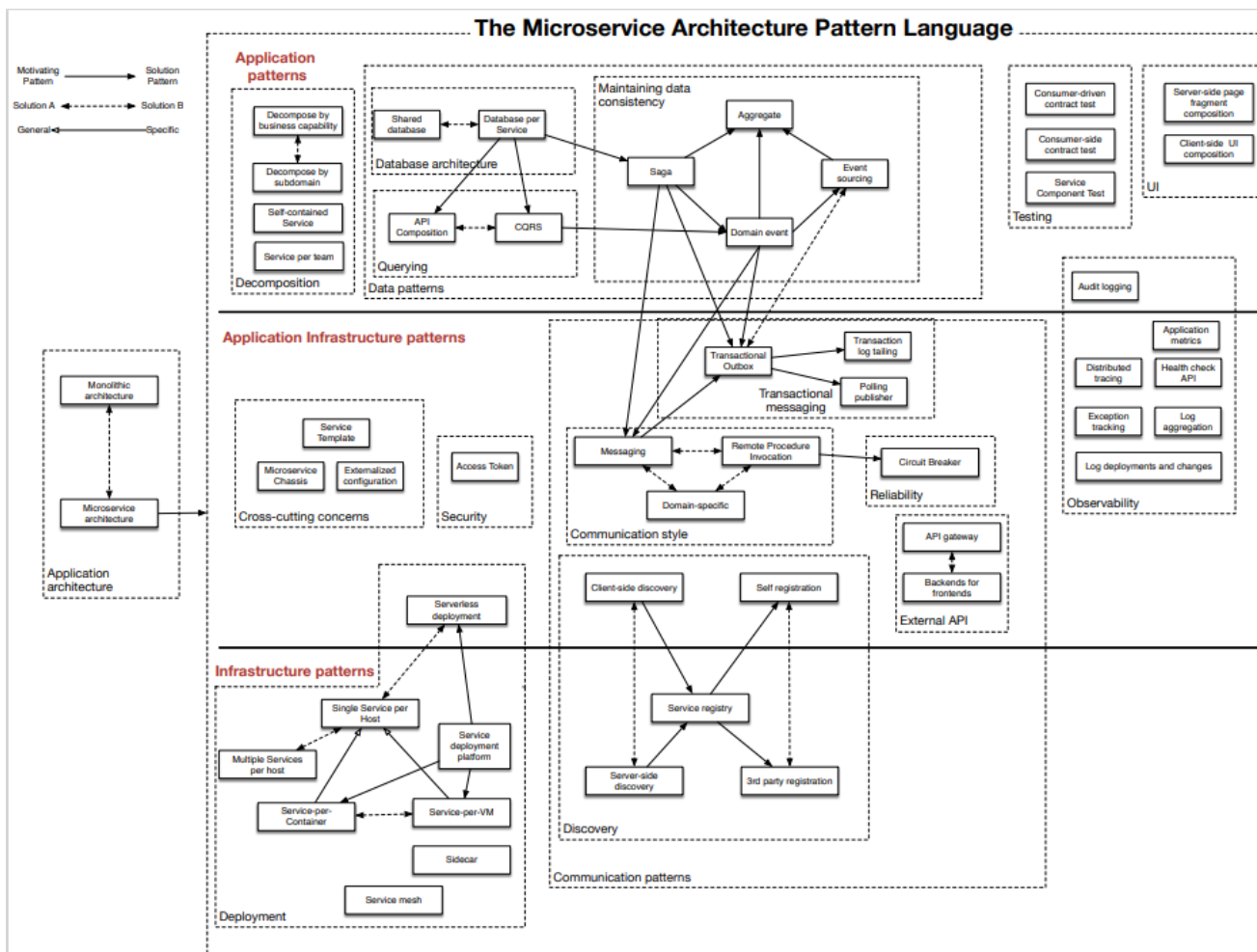
Decisões Arquiteturais

- 
- Quais são os microsserviços que compõem o sistema?
 - Como os microsserviços se comunicam entre si?
 - Como os dados devem ser distribuídos e compartilhados?
 - Como os microsserviços devem ser coordenados?
 - Como as falhas nos microsserviços devem ser detectadas, reportadas e gerenciadas?

Padrões de Projeto

- Diversos padrões de projeto são encontrados propondo soluções para diferentes problemas:
 - Comunicação distribuída
 - Dados distribuídos
 - Descoberta de serviços
 - Tolerância a falhas
 - Etc.
- Vamos explorar alguns...

Padrões de Projeto

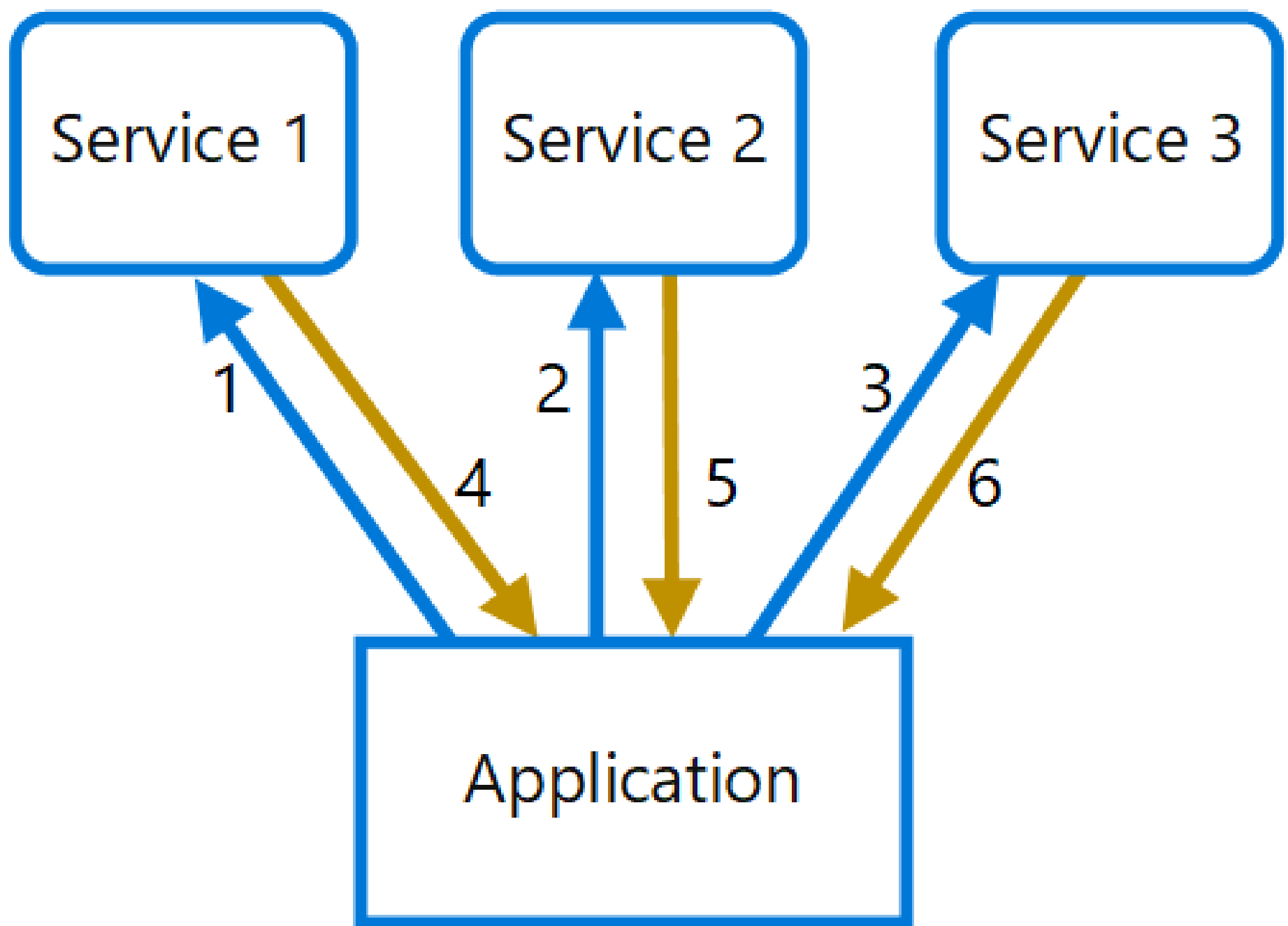


PADRÕES

API Gateway

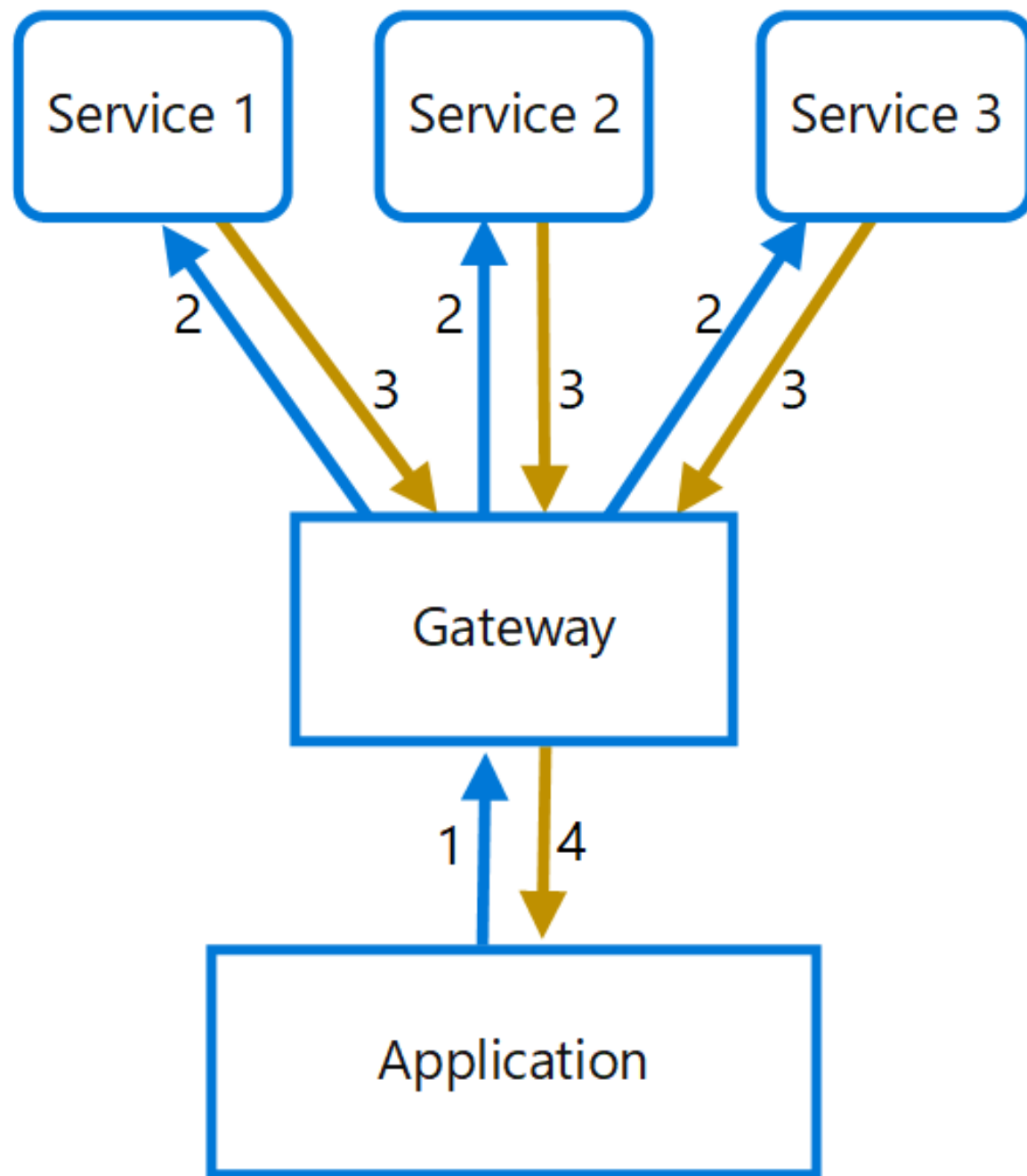
API Gateway

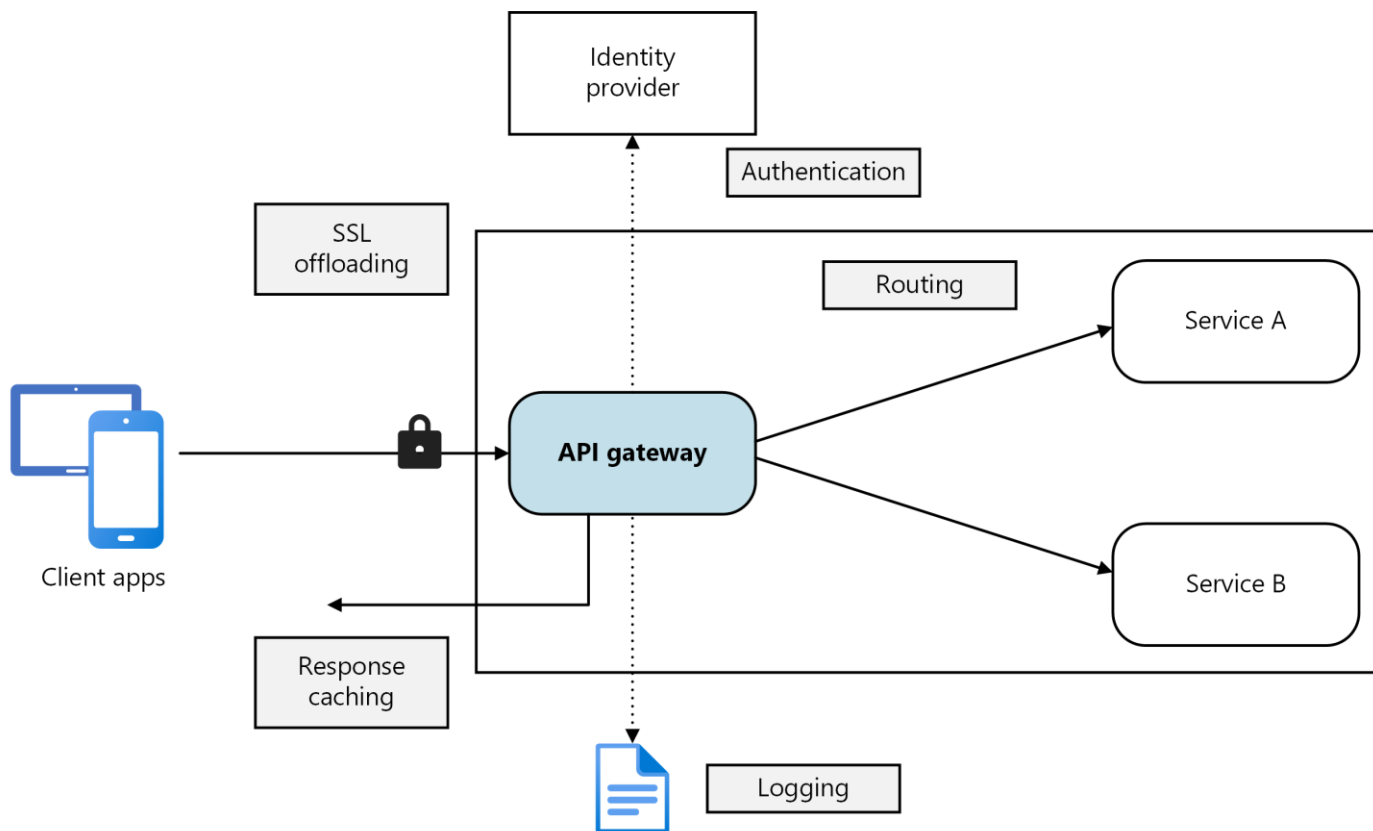
- Padrão de integração e comunicação
- Cenário:
 - Um aplicativo móvel cliente possui uma tela que necessita mostrar informações do usuário associadas a um carrinho de compras de produtos obtidos a partir de um catálogo, além de informações de identidade.
 - Deseja-se
 - Evitar uma grande troca de mensagens entre o cliente e os diferentes microsserviços que fornecem a funcionalidade de negócio.
 - Tratar de requisitos transversais como autenticação/autorização, agregação/transformação de dados, roteamento dinâmico de requisições.
 - Implementar uma fachada com protocolos “mais amigáveis” para aplicativos cliente.

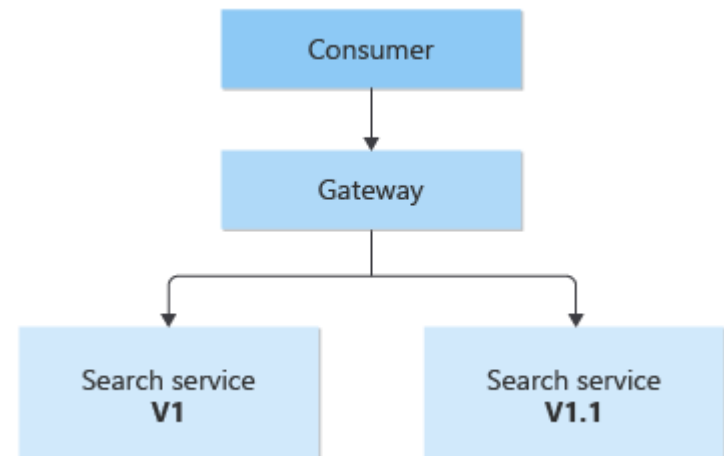
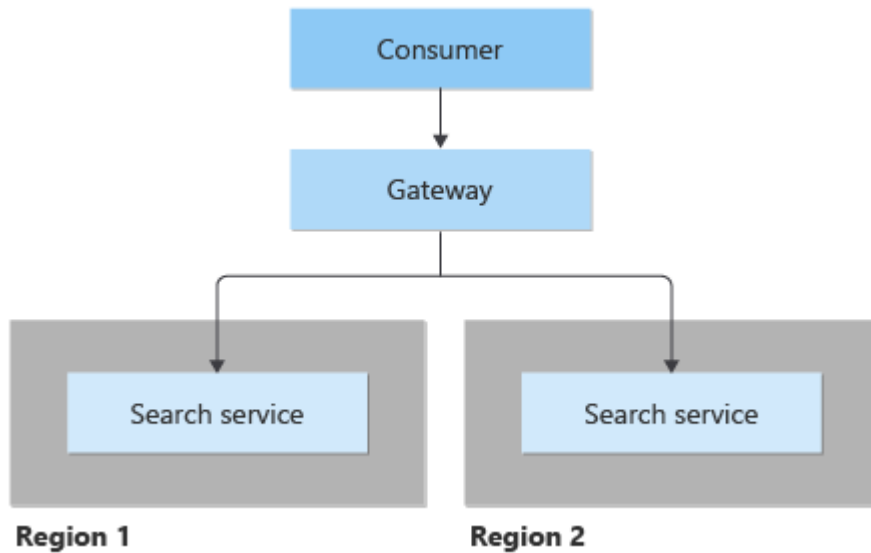
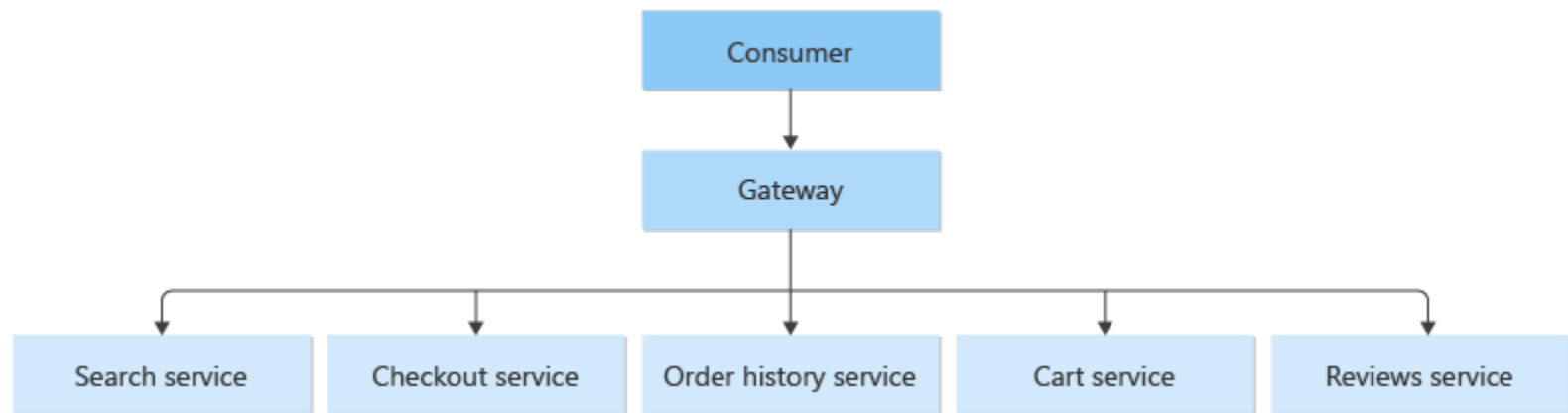


API Gateway

- Caracterização:
 - É um ponto de entrada único para comunicação entre o cliente (como o dispositivo de um usuário) e os microsserviços.
 - Ele simplifica o processo de comunicação ao abstrair as complexidades da interação com os microsserviços e permite que o cliente se comunique com eles através de um único protocolo (usualmente HTTP).
 - Ele atua como um proxy reverso, roteando solicitações para o microsserviço apropriado e lidando com tarefas como autenticação, limitação de carga, balanceamento de carga, armazenamento em cache, etc.
 - Ele pode atuar como um agregador de requisições/respostas.







API Gateway

- Saiba mais:
 - <https://microservices.io/patterns/apigateway.html>
 - <https://learn.microsoft.com/en-us/azure/architecture/microservices/design/gateway>
 - <https://docs.aws.amazon.com/apigateway/latest/developerguide/welcome.html>
 - <https://cloud.google.com/api-gateway>

API Gateway

- Implementações:
 - Spring Cloud Gateway
 - Netflix Zuul
 - <https://github.com/Netflix/zuul>
 - Amazon API Gateway
 - <https://aws.amazon.com/pt/api-gateway/>
 - Google Cloud API Gateway
 - <https://cloud.google.com/api-gateway/docs>
 - Azure Application Gateway
 - <https://learn.microsoft.com/en-us/azure/application-gateway/>
 - Kong Gateway
 - <https://konghq.com/products/kong-gateway>
 - Ocelot
 - <https://github.com/ThreeMammals/Ocelot>

API Gateway

- Suporte Spring Cloud:
 - Spring Cloud Gateway <https://spring.io/projects/spring-cloud-gateway>

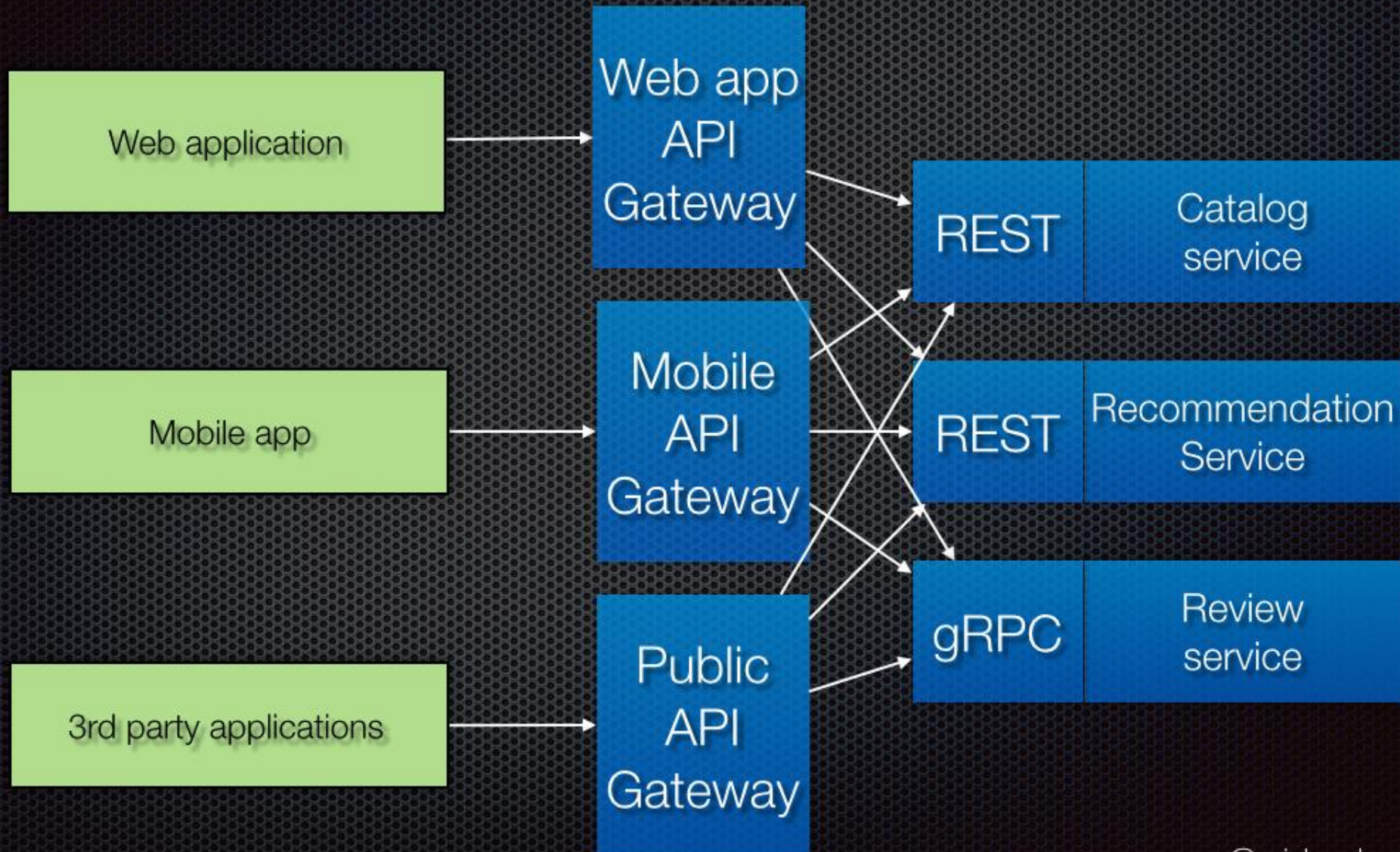
PADRÕES

BFF - Backends For Frontends

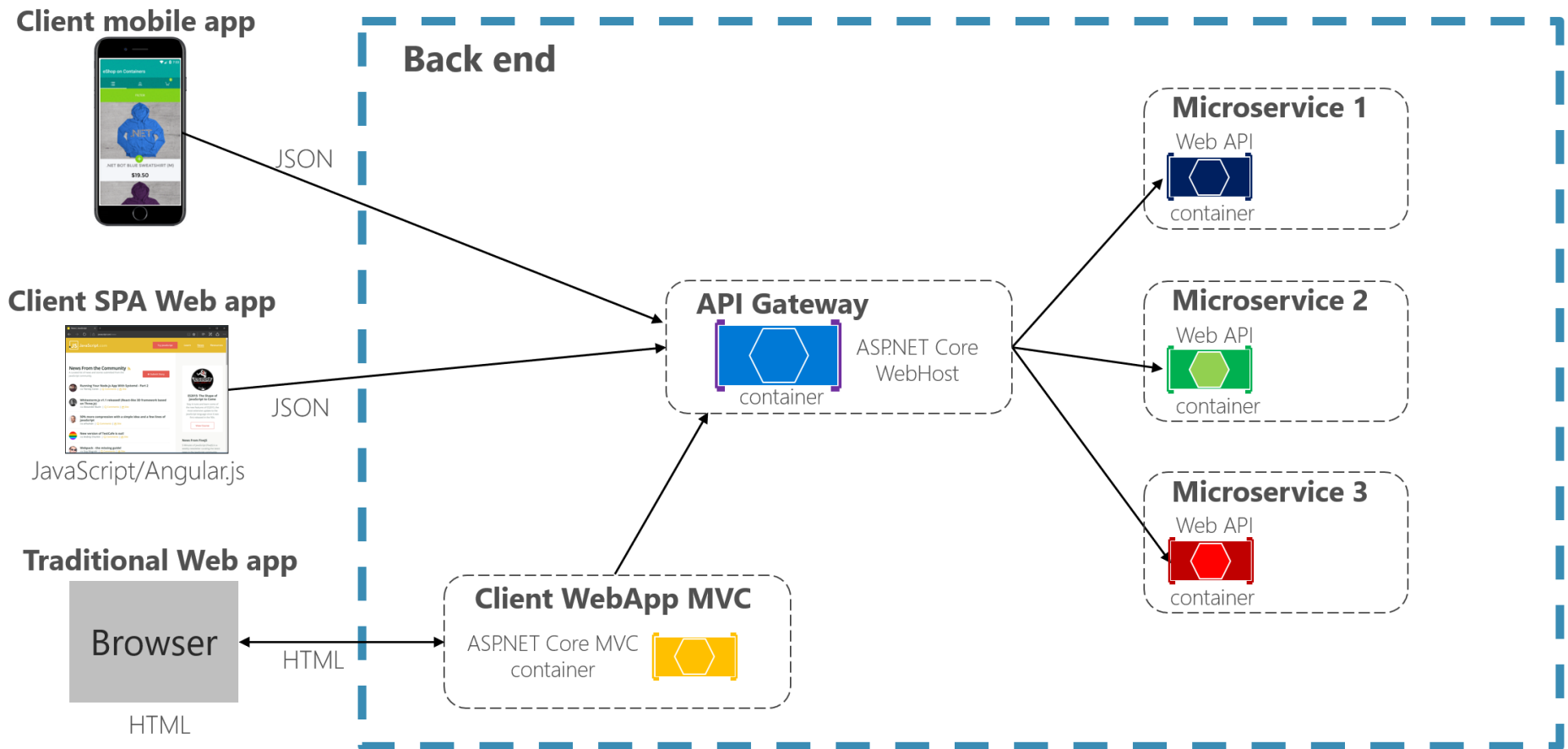
BFF

- Padrão de integração e comunicação
- Cenário:
 - Diferentes tipos de cliente irão acessar um backend de serviços.
 - Cada tipo de cliente possui algumas diferenças nos requisitos de negócio.
- Solução:
 - Aplicar uma variação no padrão API Gateway chamado de Backend For Frontends (BFF).
 - Consiste em definir API Gateways diferentes para cada tipo de cliente.

Variation: Backends for frontends



Using a single custom **API Gateway service**



Using multiple **API Gateways** / **BFF**

Client mobile app



JSON

Back end

API Gateway - Mobile



ASP.NET Core
WebHost

Microservice 1



Web API
container

Client SPA Web app



JSON

JavaScript/Angular.js

API Gateway - Web



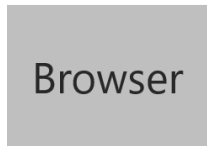
ASP.NET Core
WebHost

Microservice 2



Web API
container

Traditional Web app



Browser

HTML

HTML

Client WebApp MVC

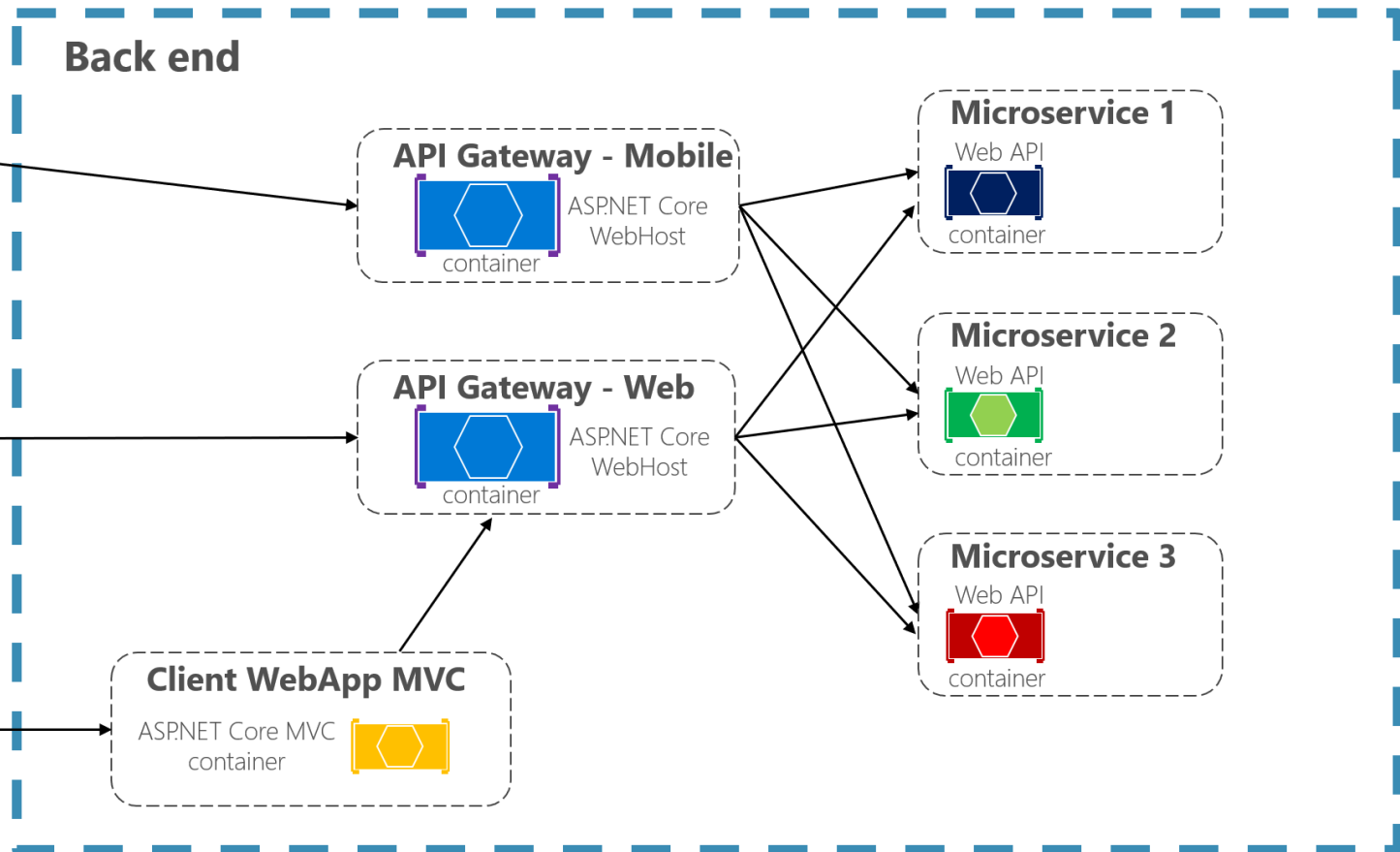
ASP.NET Core MVC
container



Microservice 3



Web API
container



BFF

- Saiba mais:
 - <https://samnewman.io/patterns/architectural/bff/>
 - <https://learn.microsoft.com/en-us/azure/architecture/patterns/backends-for-frontends>
 - <https://aws.amazon.com/pt/blogs/mobile/backends-for-frontends-pattern/>